

Test technique - Analyse de documents comptables

1. Contexte & Objectif

Comme on en a discuté, voici ton petit test technique ! L'idée c'est de concevoir un pipeline de traitement de données en Python. Tu vas devoir créer un script capable de lire, normaliser et extraire des infos à partir de documents PDF un peu différents les uns des autres (ici, ce seront de faux documents comptables).

Le but pour moi n'est pas d'avoir un code parfait du premier coup, mais de voir comment tu réfléchis, comment tu structures un projet à partir de zéro, ta logique, et comment tu te débrouilles pour chercher des solutions (avec git, la doc, Docker, etc.).

Tu as 2 à 3 semaines pour faire ça tranquillement chez toi. Prends le temps de faire les choses proprement !

2. La Data

Je vais te fournir un lot de documents PDF comptables fictifs (des factures, des petits bilans, etc.). *Petite précision : Certains seront du vrai texte copiable, d'autres seront sûrement des images/scans. À toi de voir comment gérer ça !*

3. Ce que j'attends de toi

Palier 1 : La lecture & Normalisation

Ton script doit pouvoir ouvrir et lire le contenu des fameux PDFs. - À toi de trouver la ou les bonnes librairies Python pour manipuler ça. - Comment gères-tu la différence entre un PDF natif avec du texte et un document scanné ?

Palier 2 : L'Extraction (La logique)

Une fois le texte récupéré, ton programme doit fouiller dedans et extraire des infos structurées (ex: Numéro de facture, Date, Montant HT, Montant TTC, Nom de la boîte...). - Le résultat doit finir dans un fichier de sortie propre (genre **resultats.json** ou **resultats.csv**). - **Petite contrainte** : Je préfère que tu n'utilises pas d'API type ChatGPT/Claude pour faire l'extraction magiquement. Je veux vraiment voir *ta* logique à toi (expressions régulières, algos de recherche, parcours de chaînes, etc.).

Palier 3 : L'Environnement et Docker

Il faut que ton code soit reproductible (on évite le "pourtant ça marchait chez moi !"). - N'oublie pas le petit fichier **requirements.txt**. - Surtout, tu dois conteneuriser toute l'application avec **Docker** (un **Dockerfile** qui permet de lancer le script direct).

Les Bonus (Si t'es chaud et que t'as le temps)

Si tu veux aller plus loin : - **Git avancé** : Fais-moi un historique de commits super propre, et pourquoi pas utiliser des branches. - **CI/CD** : Tu peux mettre en place de l'intégration continue (GitHub Actions ou GitLab CI) pour lancer un linter (flake8, ruff) ou pour vérifier que le conteneur Docker *build* bien à chaque nouveau commit. - **Une API** : Mets ton script derrière une mini-API web (FastAPI ou Flask) avec une route pour uploader un PDF et recevoir le JSON en retour.

4. Les Livrables (Ce que tu dois me rendre)

1. **Un repo Git public** (GitHub ou GitLab) avec tout le projet.
2. **Ton code source** commenté là où c'est utile.
3. **Un vrai bon fichier README.md**. C'est super important pour moi !

Dans le **README.md**, je dois y trouver : - Les instructions pour lancer ton projet en deux secondes avec Docker. - **Tes choix techniques** : Pourquoi telle librairie et pas une autre ? T'as benchmarké d'autres trucs ? - **Tes difficultés** : Raconte-moi ce qui a été galère. Comment tu t'en es sorti ? Et qu'est-ce que tu aurais fait de mieux avec plus de temps ?

5. Comment on évalue ça ?

Te prends pas trop la tête, l'idée c'est de voir ta façon d'aborder un problème technique !

Allez bon courage, amuse-toi bien sur le projet et on en reparle bientôt !